

Neural Ordinary Differential Equations

Corduneanu-Huci Maria, Haltrich Thanika

[GitHub Repository](#)

March 2026

Contents

1	Introduction	2
2	Main Idea	2
3	Ordinary Differential Equations	3
4	Numerical Methods for Solving ODEs	3
4.1	Euler Methods	3
4.1.1	Explicit Euler Method	3
4.1.2	Implicit Euler Method	3
4.2	Runge-Kutta Method	3
4.3	Comparision of the different methods on the CIFAR image classification problem	3
5	Neural Ordinary Differential Equations	4
5.1	Definition	4
5.2	Forward pass	4
5.2.1	Initial state	5
5.2.2	Derivative function	5
5.2.3	Integration(ODE Solver)	5
5.2.4	Output	5
6	Training Neural ODEs: The Adjoint Method	5
6.1	The Adjoint State	5
6.2	Gradient with Respect to Parameters	6
6.3	Optimal Control Theory	6
7	Comparison with Residual Neural Networks	7
7.1	ResNets vs Neural ODEs	7
7.2	ResNets as Euler Discretizations	7
7.3	Implications	7
8	Code Application	7
9	Comparision of models with image classification datasets	9
9.1	MNIST	9
9.2	CIFAR10	10
10	Limitations and Advantages	11
	References	12

Abstract

Neural Ordinary Differential Equations appeared as a shift and improvement in deep learning, replacing the discrete sequence of transformations in standard neural networks with a continuous dynamical system. Instead of parameterizing a fixed number of layers, Neural ODEs parameterize the derivative of the hidden state using a neural network. The output is obtained by numerically integrating this derivative from an initial to a final state. In this report, we focus on presenting the overall concept and methodology behind Neural ODEs. This paper highlights the relevant ODE theory, connects Neural ODEs to residual networks, analyzes the adjoint sensitivity method, discusses advantages and limitations, and presents some applications including how the N-ODE is implemented in code and how N-ODE performs relative to other neural networks on image classification tasks.

1 Introduction

In real world systems evolve continuously over time. These are developed not by discrete steps but by ongoing dynamics. Thinking about the neural dynamics in the human brain or the fluid dynamics of weather systems, these processes are typically described using Ordinary Differential Equations. The model behind these describes how systems' state changes over time. Another interesting aspect is that machine learning models, used for temporal data, operate in discrete time. For example, some models would be Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs). These models view time as a sequence of steps, updating a hidden state at each time point based on the previous one. Even though these are still effective in practice, they do not align with the nature of real-world data, in a continuous way.

This difference leads to several challenges. Discrete models usually assume that data points occur at fixed time intervals. They can also struggle to capture long-term dependencies in time without making the model much deeper and more complex. Because of these limitations, researchers have explored models that treat time as a continuous variable. This approach is captured by Neural Ordinary Differential Equations which comes with a more natural, flexible and better way to represent dynamical systems.

2 Main Idea

Standard neural network:

$h_{t+1} = \sigma(w_t h_t + b_t) \rightarrow$ each layer applies a nonlinear transformation

The hidden state changes discretely at each layer.

Residual network: $h_{t+1} = h_t + f(h_t, \theta_t) \rightarrow$ each layer adds a small transformation

This is still discrete, but the additive skip connection makes it resemble a step of a continuous process.

$h(t)$ = hidden state at layers (t)

f = neural network

θ = parameters (weights + biases)

Neural Ordinary Differential Equations generalize this to a continuous transformation:

Neural ODEs: $\frac{dh(t)}{dt} = f(h(t), t, \theta)$

$h(t)$ = hidden state continuous time (t)

f = neural network which defines the derivative of the hidden state

θ = learnable parameters

The output is computed by solving the ODE from t_0 to t_1 :

$$h(t_1) = h(t_0) + \int_{t_0}^{t_1} f(h(t), t, \theta) dt$$

This approach replaces discrete layer updates with a continuous evolution of the hidden state.

3 Ordinary Differential Equations

An ODE describes how a variable changes over time: $\frac{dy}{dt} = f(y, t)$

$y(t)$ represents the state of the system at time t , and $f(y, t)$ defines how the state changes.

Based on an initial condition $y(t_0)$, the solution of the ODE describes the trajectory of the system over time. In many cases, the solution is computed numerically using methods such as Euler or Runge-Kutta.

4 Numerical Methods for Solving ODEs

We now introduce the different numerical methods for solving ODEs. These methods are the workhorse behind how ResNet and N-ODE work during the feed forward process.

4.1 Euler Methods

The Euler methods are the most basic ODE solver methods. This method can be applied to both ResNet and N-ODE. The downside of these methods is that the order of convergence is one, meaning the error decreases linearly with the step size.

4.1.1 Explicit Euler Method

$$y_{n+1} = y_n + hf(t_n, y_n)$$

h = step size

$f(t_n, y_n)$ = derivative of the system

This method evaluates the slope at the current point and moves forward in that direction. The update rule closely resembles the update used in Residual Networks, where each layer adds a small transformation to the hidden state.

4.1.2 Implicit Euler Method

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Unlike the explicit Euler method, the derivative is evaluated at the next time step, which makes the method more stable and suitable for stiff ODEs. However, it requires solving a nonlinear equation at every step, making it computationally expensive. For this reason, it is generally not used in neural network architectures such as ResNets or Neural ODEs.

4.2 Runge-Kutta Method

This method is mainly applied to Neural ODEs. The order of convergence for this method ranges from 2 to 4. The ability to use this method in N-ODE is what makes this neural network more promising than ResNet at producing better results.

4.3 Comparison of the different methods on the CIFAR image classification problem

To be able to illustrate the differences between these numerical solvers, we decided to compare the ResNet model with the N-ODE with explicit Euler and Runge-Kutta solvers from order 2 to 4. We had expected the results to be that ResNet and N-ODE with explicit Euler performed similarly, and the accuracy of the N-ODE would improve when we use a higher order RK solver. This was not the result however:

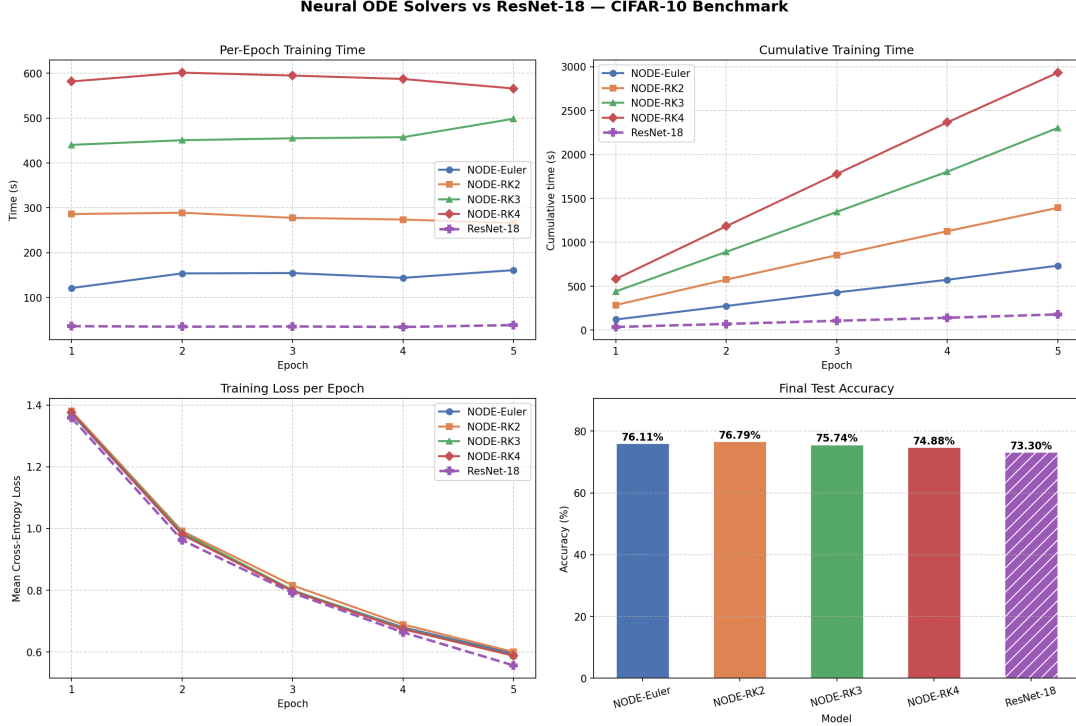


Figure 1: Comparison of Resnet, and N-ODE with different solvers on CIFAR10 dataset

In figure 1, ResNet’s performance is comparable to N-ODE. Interestingly, the RK solver with order 2 convergence performed the best, closely followed by the N-ODE with explicit euler solver. However, as we increase to RK order 3 and 4, the model performed worse but had a higher training time.

5 Neural Ordinary Differential Equations

5.1 Definition

A Neural Ordinary Differential Equation is a continuous-depth model in which the hidden state $h(t) \in \mathbb{R}^d$ follows the initial value problem:

$$\frac{dh(t)}{dt} = f_{\theta}(h(t), t), \quad h(t_0) = x$$

x = the input

f_{θ} = neural network parameterized by θ

t = from t_0 to t_1 (continuous interval)

The output of the model is the hidden state at the final time t_1 , which is obtained by numerically integrating the differential equation:

$$h(t_1) = \text{ODESolve}(f_{\theta}, h(t_0), t_0, t_1)$$

The function `ODESolve` is treated as a black box during the forward pass. In practice, any numerical ODE solver can be used. The choice of solver does not change the mathematical definition of the model but only affects the computational approximation.

5.2 Forward pass

Forward pass describes how the hidden state is actually computed in practice.

5.2.1 Initial state

The input x is treated as the initial hidden state: $h(t_0) = x$

5.2.2 Derivative function

The neural network $f_\theta(h(t), t)$ defines the derivative of the hidden state:

$$\frac{dh(t)}{dt} = f_\theta(h(t), t), \quad h(t_0) = x$$

It determines the instantaneous change of the hidden state along the continuous trajectory.

5.2.3 Integration(ODE Solver)

The forward pass of a Neural ODE is defined as: $h(t_1) = h(t_0) + \int_{t_0}^{t_1} f(h(t), t, \theta) dt$

This integral represents the continuous evolution of the hidden state from t_0 to t_1 .

The ODE solver is introduced to approximate the integral. So, ODE solver is used instead of computing the integral: $\text{ODESolve}(f_\theta, h(t_0), t_0, t_1)$

This is a numerical method that produces an approximate solution to the integral

5.2.4 Output

What is returned is the hidden state $h(t_1)$. The next step is the application of a loss function (see section 6)

6 Training Neural ODEs: The Adjoint Method

Training a Neural Ordinary Differential Equation requires computing the gradient of a loss function L with respect to the model parameters (θ) . The loss typically depends on the final hidden state $h(t_1) \Rightarrow L(h(t_1))$.

The approach is to differentiate through every step of the numerical ODE solver using reverse-mode automatic differentiation. But, this approach has a major drawback. During the forward pass, the solver may evaluate the function f_θ many times, especially when adaptive step sizes are used. If all intermediate states $h(t)$ must be stored for backpropagation, the memory cost becomes proportional to the number of solver evaluations. For complex problems or stiff ODEs, this can require a very large amount of memory.

To avoid this issue, Neural ODEs use the Adjoint Sensitivity Method. This technique was developed in the optimal control theory(see section 6.3). The key idea is to compute gradients by solving another differential equation instead of storing all intermediate states.

6.1 The Adjoint State

To compute gradients efficiently, we introduce the adjoint state $\rightarrow a(t)$. The adjoint state represents the sensitivity of the loss with respect to the hidden state at time t :

$$a(t) = \frac{dL}{dh(t)}$$

$a(t)$ tells us how changes in the hidden state at time t affect the final loss

It can be shown that the adjoint state also satisfies its own differential equation. By applying the chain rule and differentiating the original ODE, we obtain the adjoint ODE:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f_\theta(h(t), t)}{\partial h(t)}$$

An important property of this equation is that it runs backward in time. The integration starts at the final time t_1 and moves toward the initial time t_0 . The initial condition for this backward integration

is:

$a(t_1) = \frac{dL}{dh(t_1)} \rightarrow$ which is simply the gradient of the loss with respect to the output of the Neural ODE.

6.2 Gradient with Respect to Parameters

In addition to computing gradients with respect to the hidden state, we also need the gradient of the loss with respect to the model parameters θ .

This gradient accumulates along the continuous trajectory of the hidden state:

$$\frac{dL}{d\theta} = - \int_{t_1}^{t_0} a(t)^T \frac{\partial f_{\theta}(h(t), t)}{\partial \theta} dt$$

This integral is computed during the backward ODE solve. The system AUGMENTED tracks both the adjoint state and the accumulating parameter gradients while integrating backward in time.

Combining with what was presented in section 5.2, the training steps are presented below.

Training:

- Forward pass: Solve the Neural ODE forward in time from t_0 to t_1 to obtain the final state $h(t_1)$.
Compute the loss: $L(h(t_1))$
- Initialize adjoint: Compute $a(t_1) = \frac{dL}{dh(t_1)}$ using automatic differentiation of the loss
- Backward pass: Solve an augmented ODE backward from t_1 to t_0 that simultaneously tracks $h(t)$, the adjoint state $a(t)$, and the accumulated gradient $\frac{dL}{d\theta}$
- Extract gradients: At the end of the backward integration we obtain $a(t_0) = dL/dh(t_0)$ and the final parameter gradient $\frac{dL}{d\theta}$

The important insight is that the backward pass itself is another ODE solve. Because the method only needs the current state of the augmented system, the memory requirement is constant $\Rightarrow O(1)$

It is very important to take into consideration that this complexity is regardless of how many function evaluations were used in the forward pass.

But, this approach introduces a computational tradeoff. The backward integration roughly doubles the total computation time, and numerical errors may appear if the forward dynamics are not perfectly reversible.

6.3 Optimal Control Theory

The adjoint method used in Neural ODEs is closely related to techniques from classical optimal control theory. In optimal control, the idea is to minimize an objective function while a dynamical system evolves according to a differential equation.

A central result in this field is the Pontryagin Maximum Principle, which introduces an auxiliary variable called the costate or adjoint variable. This variable satisfies a backward differential equation that is mathematically identical to the adjoint equation used when training Neural ODEs.

From this perspective, the parameters of the neural network play a role similar to control variables, and the gradient $dL/d\theta$ corresponds to the derivative of a Hamiltonian function with respect to those parameters.

This connection is important because it allows Neural ODEs to benefit from decades of theoretical research in optimal control, including results on stability, convergence, and the behavior of dynamical optimization systems.

7 Comparison with Residual Neural Networks

In a standard L -layer residual network, the hidden state evolves through a discrete sequence $h_1, h_2, \dots, h_L$. The depth of the network is therefore a discrete architectural hyperparameter fixed during model design. During backpropagation, memory usage scales as $O(L)$ because the activations from all layers must be stored.

In contrast, a Neural ODE specifies only the instantaneous dynamics f_θ . The notion of depth becomes continuous, and the effective depth is determined by the integration interval $[t_1, t_0]$ and the step size chosen by the numerical solver. Memory usage during training can be reduced to $O(1)$ using the adjoint method. However, each forward and backward pass requires solving an ODE, which may involve many function evaluations when the dynamics are complex.

7.1 ResNets vs Neural ODEs

$h_{t+1} = h_t + f(h_t)$ - this is Euler discretization of an ODE.

The connection between Residual Networks (ResNets) and Neural ODEs is fundamental. It shows that many modern deep learning architectures can be interpreted as numerical ODE integration schemes, and that Neural ODEs naturally arise as the continuous limit of these models.

7.2 ResNets as Euler Discretizations

Consider a residual network with a uniform step size ($h = 1$):

$$h_{t+1} = h_t + f(h_t, \theta_t)$$

This update rule is equivalent to the explicit Euler method applied to the ordinary differential equation

$\frac{dh}{dt} = f(h, t, \theta) \rightarrow$ with step size 1 and parameters for each layer (θ_t). If the step size becomes smaller and the number of layers increases, the discrete sequence of hidden states approaches a continuous trajectory. In the limit as $h \rightarrow 0$ (infinitely many layers with infinitely small step size) $\Rightarrow$ it can be described by the continuous equation

$\frac{dh(t)}{dt} = f(h(t), t, \theta) \rightarrow$ which corresponds to the Neural ODE formulation. Therefore, a ResNet can be interpreted as an explicit Euler discretization of a Neural ODE, while a Neural ODE represents the continuous depth limit of a residual network.

7.3 Implications

- ResNets work well because the skip connections allow the hidden state to change smoothly, similar to a small step in a continuous process
- With Neural ODEs, we can let the solver decide how many effective steps (or layers) are needed, instead of manually choosing the number of layers

We can develop a training strategies, for example instead of manually choosing the number of layers, we can define a desired integration accuracy and allow an adaptive ODE solver to determine the effective depth automatically

8 Code Application

From the above sections, we have explained the conceptual differences between NODE and ResNet. Highlighting the main difference again: in ResNet, each layer i in the network outputs a vector that predicts the change from time step $i-1$ to i . Therefore in ResNet we get *discrete* layers, each symbolizing change in a time step. For NODEs however, we would like to predict the unknown variable's state at any time step t given the initial state (we are essentially solving an IVP). Therefore, the output of the neural network would be the *entire* vector space. Translating this into code means there are some key

differences in how the NODE is implemented compared to other neural networks, which is what this section will highlight.

- **what even is a continuous layer?**

when implementing neural networks, one can expect to see:

```
class MLP(torch.nn.Module):

    def __init__(self, num_features, num_classes):
        super(MLP, self).__init__()

        self.linear_1 = torch.nn.Linear(num_features, 128)
        self.linear_2 = torch.nn.Linear(128, 64)
        self.linear_3 = torch.nn.Linear(64, num_classes)
```

or

```
self.conv_1 = torch.nn.Conv2d(in_channels=1,...)

self.pool_1 = torch.nn.MaxPool2d(kernel_size=(2, 2),...)

self.conv_2 = torch.nn.Conv2d(in_channels=8,...)
```

in these snippets, we are initializing discrete layers, and we know that our training data gets passed on through each layer, allowing the model to learn different weights and biases. This is entirely different in NODE. Here, in the forward pass, we are simply calling an ODE solver for each time step:

```
class ODEAdjoint(torch.autograd.Function):
    @staticmethod
    def forward(ctx, z0, t, flat_parameters, func):
        '''calling the ode solver for each time step'''
        assert isinstance(func, ODEF)
        bs, *z_shape = z0.size()
        time_len = t.size(0)

        with torch.no_grad(): #we don't save gradients
            z = torch.zeros(time_len, bs, *z_shape).to(z0)
            z[0] = z0
            for i_t in range(time_len - 1): # --(1)
                z0 = ode_solve(z0, t[i_t], t[i_t+1], func)
                z[i_t+1] = z0

        ctx.func = func
        ctx.save_for_backward(t, z.clone(), flat_parameters)
        #ctx is what is saved and will be used for backprop later
        return z
```

–(1) is where we solve the ODE for each time step. This is the essence of the feed forward in NODE and is what is meant by 'continuous layer'

- **backpropagation with adjoint state**

In essence, using the adjoint state $a(t)$ is simply a way that allows us to perform back propagation in a 'continuous layer'. We do this by solving IVP where $a(t)$ is the variable we want to analyse:

$$\frac{da(t)}{dt} = -a(t) \frac{\partial f(z(t), t, \theta)}{\partial z} \quad (1)$$

By seeing how $a(t)$ changes through time (in this case backwards in time since we are doing back propagation), we are able to compute the gradients

$$\frac{\partial L}{\partial z(t_0)}, \frac{\partial L}{\partial \theta}, \frac{\partial L}{\partial t_0}$$

which will allow us to optimize the model!
This is done in code in the following way:

```
def forward_with_grad(self, z, t, grad_outputs):
    """Compute f and a df/dz, a df/dp, a df/dt"""
    batch_size = z.shape[0]

    out = self.forward(z, t)

    a = grad_outputs
    adfdz, adfdt, *adfdp = torch.autograd.grad( # --(1)
        # gradients needed to find da_aug/dt using the adjoint state
        (out,), (z, t) + tuple(self.parameters()), grad_outputs=a,
        allow_unused=True, retain_graph=True
    )
```

first, we define a function that computes the gradients

$$\frac{da_{\text{aug}}}{dt} = - \begin{bmatrix} \frac{\partial f}{\partial z} & a \frac{\partial f}{\partial \theta} & a \frac{\partial f}{\partial t} \end{bmatrix}$$

(this is done in --(1))

Then, as we solve our IVP $\frac{da(t)}{dt}$, we generate the gradients for each time step (done in --(2)):

```
with torch.set_grad_enabled(True):
    # we are tracking gradients again since we will need it for backprop
    t_i = t_i.detach().requires_grad_(True)
    z_i = z_i.detach().requires_grad_(True)

    func_eval, adfdz, adfdt, adfdp = # continue next line
    = func.forward_with_grad(z_i, t_i, grad_outputs=a) # --(2)
    adfdz = adfdz.to(z_i) if adfdz is not None # cont next sentence
    else torch.zeros(bs, *z_shape).to(z_i)
    adfdp = adfdp.to(z_i) if adfdp is not None # cont next sentence
    else torch.zeros(bs, n_params).to(z_i)
    adfdt = adfdt.to(z_i) if adfdt is not None # cont next sentence
    else torch.zeros(bs, 1).to(z_i)
```

These gradients are then used to compute $\frac{\partial L}{\partial z_0}$ and then used to adjust adjoints:

```
# Adjust adjoints
adj_z += dLdz_0
adj_t[0] = adj_t[0] - dLdt_0
return adj_z.view(bs, *z_shape), adj_t, adj_p, None
```

and we carry on training and adjusting adjoints until we have optimized our model to a certain extent like in normal backpropagation.

9 Comparison of models with image classification datasets

9.1 MNIST

We decided it would be insightful to see how NODE performs compared to CNN, MLP, and ResNet. Therefore, we decided to train the models on the MNIST dataset to measure the overall performance and the time it took for training. The results are as follows:

As shown in Figure 2, it can be seen that ResNet performed the best but has a longer training time. And from Figure 3, it can be seen that N-ODE performed equally good as ResNet, though it has a significantly longer training time. We hypothesize that this is due to the simplicity of the dataset and therefore performed another comparison:

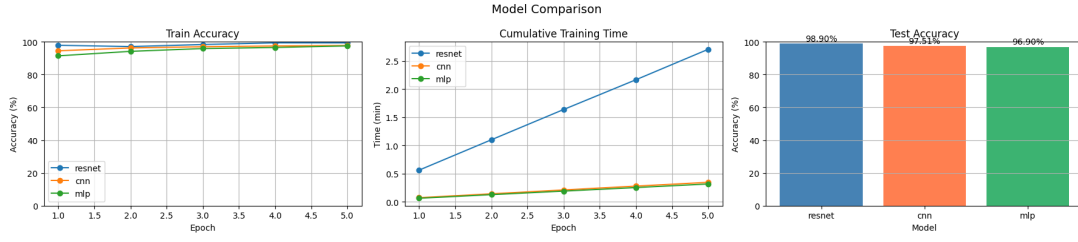


Figure 2: Comparison of Resnet, CNN, and MLP on MNIST dataset

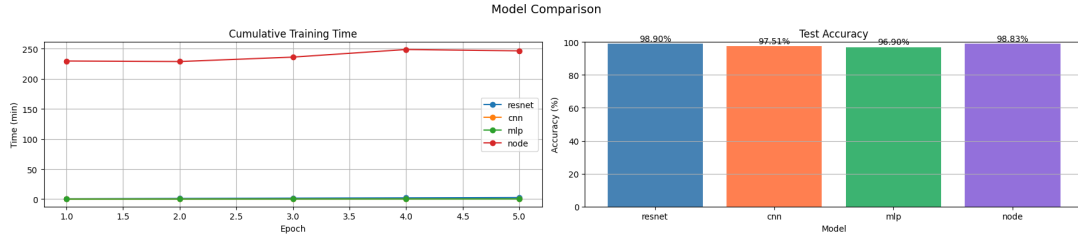


Figure 3: Comparison of Resnet, CNN, MLP, and N-ODE (using 2-order Runge-Kutta Solver) on MNIST dataset

9.2 CIFAR10

The CIFAR-10 dataset consists of 60000 32x32 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images. This dataset offers more complexity than the MNIST data. The result when applying the different models to this dataset is as follows:

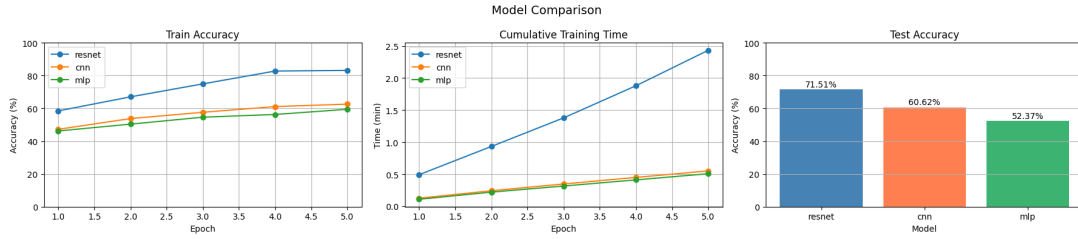


Figure 4: Comparison of Resnet, CNN, and MLP on CIFAR10 dataset

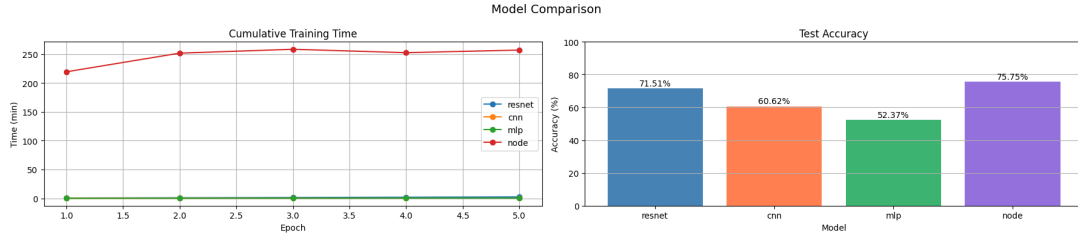


Figure 5: Comparison of Resnet, CNN, MLP, and N-ODE (using 2-order Runge-Kutta Solver) on CIFAR10 dataset

From Figure 4 and Figure 5, it can be seen that as we use more complex data, the power ResNet and N-ODE is more visible. However, similar to the previous case, the accuracy of ResNet and N-ODE are comparable but the training time for N-ODE is significantly higher.

10 Limitations and Advantages

From section 9, the **limitations** of N-ODE become visible. That is, the training time of this model is significantly higher than the other neural networks, but the accuracy is similar to that of ResNet. However, N-ODE have **advantages**, which are the following:

- the flexibility to change numerical integrators.
 - we can choose different methods talked about in section 4. These different integration methods have different advantages and tradeoffs, i.e., accuracy, stability, and computation time
 - the possibility to use Symplectic Integrators which are designed for solving Hamiltonian systems that conserve the symplectic structure of phase space.
- allows for irregularly spaced data for training
 - useful when we do not have access to regularly spaced data
 - can be used to obtain regularly spaced data which can then be applied to other models to help interpret the ODE (since the continuous layer in the N-ODE is a black box)
- Due to how back propagation is performed in N-ODE, the memory complexity during training is just $O(1)$

References

- [1] M. Surtsukov, *Neural Ordinary Differential Equations*, 2019. Available online: <https://msurtsukov.github.io/Neural-ODE/>
- [2] A. Krizhevsky, V. Nair, and G. Hinton, *The CIFAR-10 and CIFAR-100 Datasets*, 2009. Available online: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [3] YouTube, *Neural ODEs (NODEs) [Physics Informed Machine Learning]*. Available online: <https://www.youtube.com/watch?v=nJphsM4ob0k>
- [4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, *Neural Ordinary Differential Equations*, Advances in Neural Information Processing Systems, vol. 31, 2018. Available online: <https://papers.nips.cc/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html>
- [5] P. Kidger, *On Neural Differential Equations*, 2025. Available online: <https://www.sciencedirect.com/science/article/pii/S2950550X25000263>
- [6] R. Amos et al., *Neural ODEs: Implicit Layers Tutorial*, 2025. Available online: https://implicit-layers-tutorial.org/neural_odes/
- [7] J. Doe et al., *Advances in Neural Differential Equations*, Communications Biology, 2026. Available online: <https://www.nature.com/articles/s42003-026-09758-w>